

The Internal Logic of Symulator

Symbolic Simulation of Linear Electric Circuits
by Staged Solution of Unknowns

Roberto Perez-Franco

Claude (Fable 5)

*The purpose of circuit analysis is insight,
not mathematics.*

— after R. W. Hamming [4]

Abstract

Symbulator is a symbolic simulator of linear electric circuits, developed continuously since 1999, first for Texas Instruments graphing calculators and, since 2026, as an open-source Python library built on the SymPy computer algebra system. Given a plain-text circuit description and an analysis type—direct current (DC), alternating current (AC), complex frequency domain (FD), or transient (TR)—it returns exact symbolic or numeric expressions for every node voltage, branch current, voltage drop, and power in the circuit.

This monograph documents the solver’s internal logic. A history chapter traces the program from its 1999 origins on the TI-89 to the 2026 computer-algebra port. The core chapter explains, first in words and then in thoroughly annotated pseudo-code, how a DC analysis proceeds: parsing and validation; the *stamping* of each element into a system of equations; and the staged solution of unknowns by *degree*—first-degree unknowns (node voltages, and the currents of current sources) obtained directly, second-degree unknowns (the remaining branch currents) obtained with or after them, and third-degree quantities (voltage drops, powers, and the resistance or impedance seen by each source) derived once the system is solved. A chapter of variations covers how AC, FD, and TR analyses differ from the DC template, the historical *Impala* mode, and the expert mode, including multiple solutions. A further chapter presents the equivalence tools (Thévenin/Norton, equivalent resistance, two-port extraction, and gain) as thin orchestrations of the same engine. The monograph closes with worked exemplars of difficult circuits, and with the voices of Symbulator’s users across more than two decades.

Contents

1	A history of Symbolator	3
1.1	Before Symbolator	3
1.2	Two decisions	3
1.3	The calculator years: versions 1 to 5 (1999–2001)	4
1.4	The long pause, and version 6 (2013)	4
1.5	Versions 7 and 8 (2023)	4
1.6	Version 9: the computer-algebra port (2026)	5
2	The logic of Symbolator	6
2.1	The idea in words	6
2.2	The input language	7
2.2.1	Circuit descriptions	7
2.2.2	The value language	7
2.2.3	Validation	8
2.3	Notation for the pseudo-code	8
2.4	DC analysis, annotated	9
2.4.1	Stamping	9
2.4.2	Closing references to known answers	11
2.4.3	Solving by degrees	11
2.4.4	Diagnosing the unsolvable	12
3	Variations on the template	13
3.1	Alternating current	13
3.2	Frequency domain	13
3.3	Transient analysis: a Laplace round trip	14
3.4	The Impala mode	14
3.5	Expert mode	15
3.6	Multiple solutions and their ranking	16
4	Tools on top of the template	17
4.1	Thévenin and Norton equivalents	17
4.2	Equivalent resistance of a passive network	17
4.3	Two-port parameter extraction	18
4.4	Small tools	18
4.5	What the port to version 9 changed	18
5	Exemplars	20
5.1	The two-stage amplifier of 1999	20
5.2	One of each: the 2013 showcase	20
5.3	Prof. Eliane Boulet’s switching transient	21
5.4	Three-phase: wye–delta with line impedances	22
5.5	The ideal transformer, symbolically	22
5.6	Coupled coils with initial conditions	23
5.7	Two op-amps, all symbols	23

6 The users, over the decades	24
6.1 The early years	24
6.2 The long run	24
6.3 Closing reflections	24
Acknowledgments	25
Bibliography	26

Chapter 1

A history of Symbulator

Engineering students around the world learn linear circuit analysis as part of their formation, and circuit theory asks them to master a considerable mathematical apparatus along the way. Yet circuit analysis is not—and should not be—about that apparatus: the focus belongs on understanding the circuits being analyzed. To paraphrase Hamming, *the purpose of circuit analysis is insight, not mathematics* [4]. Symbulator—a portmanteau of *symbolic simulator*—was created to act on that conviction: a program that takes over the mathematics of solving a linear circuit, symbolically and numerically, so that the student can attend to the circuit itself.

1.1 Before Symbulator

Symbulator was created by Roberto Perez-Franco, then an engineering student at the Technological University of Panama (Universidad Tecnológica de Panamá) in Azuero. The project has an unusually well-documented prehistory [1]. When circuits are solved by hand, innumerable small details—an unseen negative sign, a misplaced decimal point—separate a right answer from a wrong one; professional engineers reach for a numerical simulator such as SPICE, but most students, for academic and economic reasons, cannot do so in the classroom, and a numerical simulator cannot follow them into the symbolic problems their textbooks pose. At the start of 1999 the author set himself the goal of a linear circuit simulator that would assist students *in the classroom*, numerically and symbolically, across all four analyses of the curriculum.

The first attempts were improvements to the best calculator simulators then available, both for the HP48GX. Per Stenius’s numerical simulator *CSim* 2.61 (1991) became *CSim* 3.01 under the author’s hand, alongside two variants: *Assistant II* for novices and *ZAC* 1.4 for impedance work. From Don Robinson’s *NFAP* (1994) he derived a *Symbolic Voltage Finder* and a *Symbolic Current Finder*. Neither line satisfied: the HP48GX algorithms did not lend themselves to symbolic work, and the derivatives fell short of a complete companion for a student’s whole course of study. A new simulator was needed, on a new method and a new machine.

1.2 Two decisions

Two design decisions, made before a line of the eventual program was written, fixed its character permanently.

For the analysis method, the author weighed nodal analysis (including its modified and gyrator-transform forms) against what he called the *equation generation* method, which he had met through Joe Riel, author of *Syrup*, a symbolic circuit simulator for Maple V. Nodal analysis is powerful, and its matrices suit numerical work; but extracting answers from the final vector is awkward, and the matrices grow unwieldy in symbolic problems. Equation generation instead writes out the equations that describe the circuit’s voltages and currents—“just as a student would to solve the problem by hand” [2]—and hands them to an equation

solver. It won on user-friendliness: it gives the simplest possible notation for dependent sources, transformers, and two-ports, at the acknowledged cost of a more complex program.

For the platform, the author weighed the Hewlett-Packard and Texas Instruments lines. The HP48GX and HP49G algebra systems of the day lacked the symbolic manipulation the simulator would demand—the 49G’s `solve()` was “particularly weak and messy”—while the TI-89’s Derive-based CAS offered the strongest `solve()/cSolve()` available and, decisively, the automatic substitution of stored expressions: define a variable, and every later expression that names it resolves through it. The TI-89 also happened to be the cheapest of TI’s high-performance models. The choice would shape a generation of engineering students’ calculator purchases.

1.3 The calculator years: versions 1 to 5 (1999–2001)

Programming began on 30 March 1999, over the Holy Week break: first a minimal DC analysis accepting only resistors and independent sources; then, cloned from it, the AC, FD, and TR analyses, while the element catalog grew to essentially its present form—inductors, capacitors, op-amps, ideal transformers, mutual inductances, and the six two-port families. By the author’s own accounting, the concept took a week of thought and consultation, and the program about a hundred hours spread over ten months [1].

The young simulator spread at the speed of the early web. By November 1999 it had been downloaded more than 5,700 times from `ticalc.org` alone, with users writing from some twenty countries; a student in Texas reported twelve classmates using it in one circuits course. A paper describing it, submitted under the pseudonym *Thévenin*, won first place at the IEEE Student Paper Contest for Latin America in 2000 [1]. Version 5, also known as *Symbulator Q*, served as the author’s graduation thesis in January 2001, and was documented by *The Symbulator Book*, a complete twelve-chapter reference in Spanish whose first four chapters were translated to English by users Tim Hutcheson and Doug Burkett [2].

In these versions the circuit was described in a five-column matrix—name, two nodes, value, and a fifth column holding initial conditions (or transformer turns), zero-filled elsewhere—and each analysis was invoked as a program call, `scs\dc(...)`, whose answers were stored in calculator variables under mnemonic names: `v3` for a node voltage, `ir1` for a current, `pe` for a power. That naming, and the architecture beneath it, survive to the present day.

1.4 The long pause, and version 6 (2013)

A dozen years passed—degrees, a Fulbright, MIT—before version 6, released as a beta in June 2013 under a Creative Commons licence while the author was a researcher at MIT [3]. Version 6 replaced the matrix-style description with the string form still used today (elements separated by colons, no fill columns), retained the TI-89 as its platform, and was documented by an unfinished English edition of *The Symbulator Book* [3]. For Laplace transforms it relied, as its predecessors had, on *DiffEq* by Lars Frederiksen.

1.5 Versions 7 and 8 (2023)

Versions 7 (for the TI-89 Titanium and compatible devices such as the Voyage 200) and 8 (adapted to the TI-Nspire CX II CAS) were released in 2023, accompanied by a comprehensive documentation rewrite whose worked examples—drawn from nine widely used textbooks—now number in the hundreds. Version 7 kept Frederiksen’s *DiffEq*; version 8 uses his Laplace functions as adapted for the TI-Nspire by Philippe Fortin.

1.6 Version 9: the computer-algebra port (2026)

Version 9, released in 2026, is a port of the classic TI-Basic code to Python, with the SymPy computer algebra library supplying the symbolic mathematics that the calculators' built-in CAS supplied before. It runs anywhere Python runs—phones, tablets, laptops, notebooks, and, via WebAssembly, entirely inside a web browser—and, no longer bound by calculator memory, solves circuits faster than its predecessors. The port was carried out by the author working with Claude, Anthropic's AI assistant, completing in weeks a task that had waited two decades. Symulator has always been free of charge; since 2026 it is open source under the MIT licence, published as the `symulator` package on PyPI.

This monograph documents the solver's internal logic as a single conceptual algorithm. Where the calculator versions and the computer-algebra port genuinely differ in *mechanism* (though never in results), the difference is noted in place, and Chapter 3 collects the calculator-era mechanisms worth remembering.

Chapter 2

The logic of Symbulator

This chapter explains how Symbulator solves a circuit, using direct current as the template; Chapter 3 then presents every other analysis as a variation on this template. We proceed in words first, and in pseudo-code second.

2.1 The idea in words

Symbulator’s whole pipeline can be said in one sentence, and was, in the 1999 paper [1]: *the program reads the circuit description, writes down the equations that describe its behavior, solves them, and stores all the answers in variables with their respective names*. Everything else is the careful working-out of that sentence.

Reading means parsing a plain-text description in which each line names one element, its terminals, and its value—the value being a number, a symbol, or an expression; a dependent source is simply a source whose value names another answer. Reading ends with validation: the description must name a reference node, no element may short its own two terminals, and every node must have a conduction path to the reference.

Writing down the equations is called *stamping*: one pass over the element list in which each element contributes its own defining voltage–current relationship (Ohm’s law for a resistor, a fixed voltage for a source), and adds its current into a running per-node sum. When the pass ends, each node’s sum is set to zero: by Kirchhoff’s Current Law (KCL), the currents leaving a node must balance. One equation per element law, one per non-reference node.

Solving hands the system to the platform’s equation solver—the TI-89’s `solve()` then, SymPy’s now. But not every answer needs to be *in* the system, and this is the architectural idea Symbulator has carried since 1999: answers have *degrees*. First-degree unknowns are the node voltages, and the currents of current sources—the latter known outright, since a current source’s current *is* its value. Second-degree unknowns are the remaining branch currents: some (a voltage source’s current) must be solved simultaneously with the voltages, since nothing local determines them; others (a resistor’s, a capacitor’s) follow from the solved voltages by the element’s own law, by direct substitution. Third-degree quantities are derived after the solve, by plain algebra on the first two degrees: the voltage drop across each element, the power each element absorbs, and, for each source, the resistance the source sees looking into the rest of the circuit.

Storing the answers gives every quantity a systematic name — v_n for node voltages, i_e for currents, v_e , p_e , r_e for the derived quantities of element e — and it is by these names that dependent sources, expert-mode equations, and the user’s own follow-up questions refer to them.

The original documentation describes the same machinery operationally, in terms of *levels* [2]: the program “first generates a set of 1st level equations and a list of 1st level unknowns”—node voltages, plus the currents that must be solved simultaneously (those of voltage sources and, in DC, inductors); once solved, “2nd level expressions,” generated during the same pass, are *evaluated* rather than solved—the locally computable currents; and finally the “3rd level expressions”—the powers—are built from everything before them.

Degrees classify answers by what they are; levels, by how they are obtained. They coincide except at the seam: a voltage source’s current is a second-degree answer that the calculator had to place in the first-level *solve*, while a current source’s current is first-degree precisely because it never needs solving at all. Keeping the simultaneous set small was what made symbolic solving feasible on a late-1990s calculator; version 9, with SymPy underneath, solves the joined system in one call and keeps the staged view as what it always really was—the right way to *understand* the solver.

2.2 The input language

2.2.1 Circuit descriptions

A circuit is described as plain text, one element per line (or colon-separated, in the calculator style). Each element is a comma-separated list beginning with its name; the first letter of the name selects the element’s type. Table 2.1 gives the complete catalog.

Table 2.1: Element catalog. The first letter of the name selects the type.

Letter	Element	Fields after the name
r	resistor	n_1, n_2, value
l	inductor	$n_1, n_2, \text{value } [, i(0)]$
c	capacitor	$n_1, n_2, \text{value } [, v(0)]$
e	voltage source	n_1, n_2, value
j	current source	n_1, n_2, value
o	ideal op-amp (nullor)	n_+, n_-, n_{out}
m	mutual inductance	$L_{\text{name1}}, L_{\text{name2}}, M$
s	short circuit	n_1, n_2
t	ideal transformer	n_1, n_2, N_1, N_2
z y h g a b	grounded two-port	n_1, n_2

Sources may be *independent* (a numeric or symbolic value) or *dependent*: a value may reference any other answer by name, so `j2,3,0,0.5*i_r1` is a current source of half the current through `r1`, and `e2,2,0,3*v_rx` a voltage source controlled by the drop across `rx`. This uniform treatment—a dependent source is just a source whose value happens to mention an answer—is one of Symbolator’s simplest and most distinctive design decisions, and was among the reasons the equation-generation method was chosen in the first place: no special element types, no auxiliary defining branches, merely an expression with a name in it.

The optional trailing field on `l` and `c` is an initial condition (initial inductor current; initial capacitor voltage), read only by the FD and TR analyses and treated as zero when absent. `m` is unusual in that it couples two *elements* (named inductors, or, in AC, elements holding impedances) rather than two nodes. The six two-port letters denote a black box characterized only by its four *z/y/h/g/a/b* parameters, each port’s second terminal being implicitly ground.

2.2.2 The value language

Element values, source waveforms, expert-mode equations, and tool arguments all pass through one expression reader, which accepts:

- **Numbers and symbols**, and arithmetic over them (`+` `-` `*` `/`, exponentiation as `^` or `**`, and function calls such as `sqrt`, `exp`, `sin`). Implicit multiplication is inserted

where the calculator allowed it ($2\text{ir}3 \rightarrow 2*\text{ir}3$), and e^x is read as e^x .

- **SI prefixes** in the calculator’s quoted notation: 4.7'u is 4.7×10^{-6} , 1'k is 10^3 . A *bare* suffix (1k) is inherently ambiguous—one thousand, or one times a variable named k ?—so a policy argument decides: refuse and ask (the default), read all as SI, or read all as products.
- **Phasors** in textbook angle notation: $(110\angle-120^\circ)$. These are converted to rectangular complex numbers on entry. The conversion is deliberate: carrying $e^{j\theta}$ factors symbolically makes some mesh systems practically unsolvable, while the rectangular form solves in seconds; the cost is exactness of the angle, accepted on the view that a phasor angle is a measurement.
- **Waveform shorthands** for TR sources: $u(t)$ is the unit step (Heaviside), $\text{delta}(t)$ the unit impulse (Dirac).
- **The parallel shortcut** $[a,b,\dots]$, expanded to $\text{pr}(a,b,\dots)$, the parallel combination $(\sum_k 1/z_k)^{-1}$ with the convention that a zero branch dominates ($\text{pr} = 0$).
- **Braces** $\{\text{expr}\}$ in FD source values, meaning “this value is written in the time domain; transform it,” i.e. $\mathcal{L}\{\text{expr}\}$.

Because descriptions may come from strangers over the web, the reader is also a security boundary: expressions are parsed into a syntax tree and refused unless they are plain arithmetic, and names resolve against a small fixed namespace (so a user’s Q is a symbol, never an internal object of the algebra system).

2.2.3 Validation

Before any equation is written, the parsed element list is validated (Algorithm 1). Names and node labels are case-insensitive and fold to lower case; duplicate names are refused. Each element must have exactly the field count its type demands (one more is allowed on $1/c$, for the initial condition). The whole-circuit checks then enforce three properties that the solver depends on: a reference node exists; no two-terminal element shorts its own terminals together; and every node has a conduction path to the reference. The last check matters more than it may appear: a floating sub-circuit has no defined voltages, and without the check the solver would quietly return its voltages parametrized in one another ($v_2 = v_3$) rather than flag the mistake.

2.3 Notation for the pseudo-code

Throughout, n_1, n_2 denote an element’s two terminals as written in the description, node 0 is the reference (ground), v_n is the voltage of node n with $v_0 \equiv 0$, and i_e is the current through element e , defined positive when flowing from n_1 to n_2 *through the element*. In pseudo-code, $\mathcal{E}$ is the parsed element list, Σ the equation system under construction, $\mathcal{U}$ the list of unknowns, and $\mathcal{K}$ the map of *known* quantities (named answers that never become unknowns). Assignment is $\leftarrow$; EMIT adds an equation to Σ ; and “adds i at n_1 , $-i$ at n_2 ” abbreviates the KCL bookkeeping $\text{sum}[n_1] += i$; $\text{sum}[n_2] -= i$.

Algorithm 1 PARSE-AND-VAlIDATE: description text to element list

Input: description text D **Output:** element list $\mathcal{E}$, or an error naming the problem

```
1: split  $D$  into element strings (on newlines or colons)
2: for all element string  $x$  do
3:   expand notation shorthands in  $x$  ▷  $\angle$ ,  $[\ ]$ , quoted SI prefixes...
4:   split  $x$  on top-level commas into name and fields
5:   fold name and node fields to lower case
6:   check first letter of name is a known type
7:   check name not already used; field count fits type
8:   append Element(name, kind, fields) to  $\mathcal{E}$ 
9: end for
10: check some element touches node 0 ▷ a reference exists
11: for all  $e \in \mathcal{E}$  with two terminals do
12:   check  $n_1 \neq n_2$  ▷ no element shorts itself
13: end for
14: union-find over terminals: each element joins its nodes ▷ op-amps join all three;
   grounded two-ports join theirs to 0
15: check every node is connected to 0 ▷ no floating sub-circuit
16: return  $\mathcal{E}$ 
```

2.4 DC analysis, annotated

We now walk the full DC pipeline in pseudo-code. Everything in this section is the template; the other analyses (Chapter 3) change only the per-element stamping rules and the transforms around the solve.

2.4.1 Stamping

Algorithm 2 is the heart of the solver. Two bookkeeping conventions do quiet but important work. First, asking for a node's voltage ($v(n)$ in the pseudo-code) is what *creates* the node: it registers v_n as a first-degree unknown and opens the node's KCL sum, so even a node touched only through a voltage relation (an op-amp input, say) still ends the pass with a KCL row. Second, ground never gets a symbol: $v(0)$ is the constant 0, so the reference is built into every equation rather than solved for.

A word on each rule, in the order stamped. The *resistor* is the paradigm: one law, one new second-degree unknown, two KCL contributions. The *inductor* at DC steady state sustains no voltage—it is a short—but its current is a real answer and stays in the system. The *capacitor* at DC steady state carries no current—it is an open—so its current is recorded as the constant 0 in $\mathcal{K}$ and nothing enters the system at all. The *voltage source* fixes a voltage and leaves its current free; the *current source* fixes a current and leaves its voltage free, and because the fixed thing *is* the answer, nothing about it is ever solved: this is the sense in which current-source currents are first degree. The *op-amp* is modeled as a nullor: the two inputs are forced equal and draw no current—their absence from the KCL sums *is* the infinite input impedance—while the output contributes a free current with no return path in the model, standing for the supply rails. The *ideal transformer* relates its winding voltages by the turns ratio and its currents by the inverse ratio, so that it neither creates nor consumes power; only one winding current is a free unknown. A *two-port block* is a black box of four parameters (given numerically, or left symbolic to be referenced or solved for); whatever the family, its two defining equations are rearranged at stamp time into the form KCL wants—

Algorithm 2 STAMP-DC: element list to equation system (DC rules)

Input: validated element list $\mathcal{E}$ **Output:** system Σ , unknowns $\mathcal{U}$, knowns $\mathcal{K}$

```
1:  $\Sigma \leftarrow \emptyset$ ;  $\mathcal{U} \leftarrow \emptyset$ ;  $\mathcal{K} \leftarrow \emptyset$ ;  $\text{sum}[n] \leftarrow 0$  for all  $n$ 
2: for all  $e \in \mathcal{E}$  do
3:   switch  $\text{kind}(e)$  do
4:     case r (resistor, value  $R$ ):
5:       if  $R = 0$  then treat as s below ▷ a  $0\Omega$  resistor is a wire
6:       else
7:         EMIT  $v_{n_1} - v_{n_2} = R i_e$ ; add  $i_e$  to  $\mathcal{U}$  ▷ Ohm's law;  $i_e$  is 2nd degree
8:         adds  $i_e$  at  $n_1$ ,  $-i_e$  at  $n_2$ 
9:       end if
10:    case l (inductor):
11:      EMIT  $v_{n_1} - v_{n_2} = 0$ ; add  $i_e$  to  $\mathcal{U}$  ▷ a short at DC steady state; its current still matters
12:      adds  $i_e$  at  $n_1$ ,  $-i_e$  at  $n_2$ 
13:    case c (capacitor):
14:       $\mathcal{K}[i_e] \leftarrow 0$  ▷ open at DC steady state: no equation, no unknown
15:    case e (voltage source, value  $E$ ):
16:      if  $E = 0$  then treat as s below
17:      else
18:        EMIT  $v_{n_1} - v_{n_2} = E$ ; add  $i_e$  to  $\mathcal{U}$  ▷ + at  $n_1$ ; current left free—the source supplies whatever is demanded
19:        adds  $i_e$  at  $n_1$ ,  $-i_e$  at  $n_2$ 
20:      end if
21:    case j (current source, value  $J$ ):
22:       $\mathcal{K}[i_e] \leftarrow J$  ▷ 1st degree: the current is the value
23:      adds  $J$  at  $n_1$ ,  $-J$  at  $n_2$  ▷ no equation, no unknown; the voltage across is free
24:    case o (op-amp, terminals  $n_+, n_-, n_{\text{out}}$ ):
25:      EMIT  $v_{n_+} = v_{n_-}$ ; add  $i_e$  to  $\mathcal{U}$  ▷ the nullor: inputs equal, drawing no current
26:      adds  $-i_e$  at  $n_{\text{out}}$  only ▷ output supplies what balances its node; the missing return path is the supply rails
27:    case s (short circuit):
28:      EMIT  $v_{n_1} - v_{n_2} = 0$ ; add  $i_e$  to  $\mathcal{U}$ ; adds  $i_e$  at  $n_1$ ,  $-i_e$  at  $n_2$ 
29:    case t (ideal transformer, turns  $N_1, N_2$ ):
30:      EMIT  $v_{n_1}/N_1 = v_{n_2}/N_2$ ; add  $i_1$  to  $\mathcal{U}$  ▷ voltages by the turns ratio
31:      adds  $i_1$  at  $n_1$  and  $-i_1 N_1/N_2$  at  $n_2$  ▷ currents by the inverse ratio: power in = power out
32:    case z y h g a b (two-port, parameters  $p_{11..22}$ ):
33:      solve the family's two defining equations for  $(i_1, i_2)$  in terms of  $(v_{n_1}, v_{n_2})$  ▷ e.g.
34:      for z: invert the  $2 \times 2$  matrix
35:      add  $i_1, i_2$  to  $\mathcal{U}$ ; EMIT their defining equations; adds  $i_1$  at  $n_1$ ,  $i_2$  at  $n_2$ 
36:    end switch
37:  end for
38:  for all registered node  $n \neq 0$  do
39:    EMIT  $\text{sum}[n] = 0$  ▷ KCL: one equation per non-reference node
40:  end for
CLOSE-KNOWN-REFERENCES( $\Sigma, \mathcal{K}$ ) ▷ §2.4.2
```

port currents in terms of port voltages. Finally, every *zero-valued* voltage-defining element (0Ω , $0H$, $0V$) collapses to the short-circuit rule; a $0F$ capacitor, by contrast, is already an open circuit under the capacitor rule, with no special case needed.

2.4.2 Closing references to known answers

A dependent source may name a quantity that is *known* rather than solved for: another source's current, a capacitor's current, or an element's voltage drop (which is never an unknown—it is derived, in the third degree, as $v_{n_1} - v_{n_2}$). Left alone, such a name would enter the system as a fresh free variable that no equation constrains, and the solver would happily answer every quantity *in terms of it*—a failure disguised as a solution. The final stamping step therefore substitutes, everywhere in Σ , each known name by its stamped expression, iterating to a fixed point since one known may name another (a current source controlled by a capacitor's current is exactly that). Node-voltage names win where a node and an element share a name. After this closure, a reference like `0.5*i_cx` has become a real constraint on node voltages.

2.4.3 Solving by degrees

Algorithm 3 SOLVE-DC: the staged solution

Input: $\Sigma, \mathcal{U}, \mathcal{K}$ from STAMP-DC; expert extras if any (§3.5)

Output: ranked list of answer maps $\{\text{name} \mapsto \text{expression}\}$

```

1: EXPERT-AUGMENT( $\Sigma, \mathcal{U}, \mathcal{K}$ )                                ▷ Alg. 5, if extras given
   — 1st/2nd degree: the simultaneous system —
2:  $\mathcal{S} \leftarrow \text{solve}(\Sigma, \mathcal{U})$                             ▷ all solutions; a plain circuit has exactly one
3: if  $\mathcal{S} = \emptyset$  then
4:   DIAGNOSE-UNSOLVABLE( $\mathcal{E}$ )                                ▷ name the contradiction; §2.4.4
5: end if
6:  $\mathcal{S} \leftarrow \text{RANK}(\mathcal{S})$                                     ▷ §3.6; a single solution is untouched
7: for all solution  $\sigma \in \mathcal{S}$  do
   — remaining 1st/2nd degree: direct substitution —
8:   for all  $(\text{name} \mapsto \text{expr}) \in \mathcal{K}$  do
9:      $\sigma[\text{name}] \leftarrow \text{expr}$  with  $\sigma$  substituted in    ▷ knowns were stamped in terms of  $v_n$ 
     symbols
10:  end for
   — 3rd degree: derived quantities —
11:  for all  $e \in \mathcal{E}$  of kind e j r c l do
12:     $\Delta v \leftarrow \sigma[v_{n_1}] - \sigma[v_{n_2}]; \quad i \leftarrow \sigma[i_e]$ 
13:     $\sigma[v\_e] \leftarrow \Delta v$                                 ▷ voltage drop across  $e$ 
14:     $\sigma[p\_e] \leftarrow \Delta v \cdot i$                           ▷ power absorbed by  $e$ 
15:    if  $e$  is a source (e or j) then
16:       $\sigma[r\_e] \leftarrow \Delta v / (-i)$                         ▷ resistance the source sees looking out
17:    end if
18:  end for
19:  for all op-amp  $e$  do
20:     $\sigma[p\_e] \leftarrow \sigma[v_{n_{\text{out}}}] \cdot (-\sigma[i_e])$     ▷ power at the output node
21:  end for
22: end for
23: return  $\mathcal{S}$ 

```

Algorithm 3 completes the analysis. On the calculators, the simultaneous system of line 2 contained only the unknowns that truly required it (the first *level*), everything locally computable being deferred to the substitution loop; in version 9 the split survives only for the two always-local cases (current-source and capacitor currents), and line 2 is a single call to `sympy.solve`. Either way the third-degree round is the same plain algebra: drops, powers, and—for sources only, where the question “what does this source see?” is meaningful—the seen resistance. The answers are stored under systematic names, mirroring the calculator variables (`v2`, `ir1`, `pe`, `re`) under which the original stored them.

2.4.4 Diagnosing the unsolvable

When the solve comes back empty, the solver does not merely apologize: it looks for the two classic contradictions of linear circuit theory and names the culprit elements. A cycle in the graph of voltage-defining edges—voltage sources, shorts, zero-ohm resistors, and, in DC, inductors—pins the same voltage more than once, and the offending loop is found by union-find and reported by name (“two sources in parallel,” in the textbook phrase). Dually, a node touched only by current-defining elements—current sources and, in DC, capacitors—has its KCL fixed by constants that cannot balance, and is reported likewise. Only if neither pattern is present does a generic message stand.

Chapter 3

Variations on the template

Every other analysis, and every mode, is the DC template of Chapter 2 with a controlled variation: different stamping rules (AC, FD), transforms wrapped around the solve (TR), extra equations folded into the system (expert mode), or—in the calculator era—optimizations that kept the symbolic load within a handheld’s reach.

3.1 Alternating current

AC analysis works in phasors at a single angular frequency ω (a number or a symbol). Three stamping rules change, and the third degree grows richer:

- The *inductor* becomes an impedance: $v_{n_1} - v_{n_2} = j\omega(L i_e + \sum_{e'} M_{ee'} i_{e'})$, the sum ranging over mutually-coupled partners—one coil’s current induces voltage in the other, which is what magnetic coupling means. A reversed winding is a negative M .
- The *capacitor*’s current becomes locally computable again, now nonzero: $i_e = j\omega C (v_{n_1} - v_{n_2})$, recorded in $\mathcal{K}$ exactly as in DC. When $C = 0$ this evaluates to an open circuit on its own, with no special case.
- A *resistor* carrying a complex value is an impedance in ohms; when such elements are coupled by an $\mathfrak{m}$ whose value is also an impedance, the mutual term $M i_{e'}$ enters with no $j\omega$ factor—the value already *is* an impedance. This matches how textbooks give coupled pairs “in $j\Omega$.”

In the third degree, complex power replaces DC power: $S_e = V_e \overline{I_e}$ under RMS phasors, or $V_e \overline{I_e}/2$ under amplitude phasors, with the real part reported as the average power (named $\mathfrak{p}$ in the first convention and $\mathfrak{ap}$ in the second, to keep the distinction audible), and sources also receive the impedance z_e they see. The names i, j are reserved as the imaginary unit only here—in every other domain they remain free for the user.

3.2 Frequency domain

FD analysis stamps in the Laplace variable s , and the energy-storage elements carry their initial conditions into the equations:

$$\text{inductor: } \frac{v_{n_1} - v_{n_2}}{s} = L \left(i_e - \frac{i_e(0)}{s} \right) + \sum_{e'} M_{ee'} \left(i_{e'} - \frac{i_{e'}(0)}{s} \right), \quad (3.1)$$

$$\text{capacitor: } i_e = sC (v_{n_1} - v_{n_2}) - C v_e(0), \quad (3.2)$$

the transforms of $v = L di/dt$ and $i = C dv/dt$ with the initial state folded in. Source values are expected already written in s (a step of amplitude k is k/s), with the brace shorthand `{expr}` transforming a time-domain value on entry. No third degree is computed in FD: an s -domain power has no established meaning in the curriculum Symbulator serves, and the original made the same choice.

3.3 Transient analysis: a Laplace round trip

TR is FD wrapped in two transforms (Algorithm 4): sources are carried *into* the s -domain, the FD engine solves, and every answer is carried *back* to time. Both crossings have case analyses that are easy to get wrong and are worth stating exactly.

Algorithm 4 TR: transient analysis via the s -domain

Input: description D with time-domain sources; expert extras

Output: time-domain answer map

— *sources in: for each independent source value x* —

- 1: **if** x mentions t **then** $x \leftarrow \mathcal{L}\{x\}$ ▷ a waveform
- 2: **else if** x mentions s **then** leave x ▷ already transformed by the user
- 3: **else if** x references another answer **then** leave x ▷ a controlled source: a relation, not a waveform
- 4: **else** $x \leftarrow x/s$ ▷ a constant: a step of that amplitude
- 5: **end if**
- 6: expert equations and conditions that reference answers cross the same way, side by side
- 7: $\sigma \leftarrow$ FD solve of the converted description ▷ initial conditions live in the $1/c$ stamps
- *answers out: for each solved value y* —
- 8: **if** y mentions s **then** $\sigma[y] \leftarrow \mathcal{L}^{-1}\{y\}$, folding away a bare $u(t)$ factor
- 9: **else if** y is a solved expert unknown **then** leave y ▷ a scalar, e.g. an amplitude
- 10: **else if** y references another answer **then** leave y ▷ reads identically in s and t
- 11: **else** $\sigma[y] \leftarrow y \cdot \delta(t)$ ▷ constant in s is an impulse
- 12: **end if**

On the way in, the four-way test distinguishes a waveform (transform it), a value the user already wrote in s (leave it), a controlled source (its value is a *relation* between answers—“twice whatever that current is”—which reads identically in either domain and must not be transformed), and a bare constant (a step). On the way out, an answer still carrying s is inverse-transformed, with one cosmetic rule: a factor of $u(t)$ multiplying the whole answer is folded away, since every TR answer exists only for $t \geq 0$ anyway—but only the exact factor $u(t)$; a delayed step $u(t - t_0)$ is genuinely informative and stays. An answer *constant* in s requires the most care: a step arrives as k/s and any waveform brings its own s , so a bare constant has nowhere else to come from than an impulse, and is reported as $k \delta(t)$ —unless it is a solved expert scalar (an amplitude is a number, not a waveform) or a controlled source’s echo of its controlling answer. Answers whose inverse transform has no closed form are omitted individually rather than failing the analysis, since the rest of the circuit’s answers remain valid and useful.

3.4 The Impala mode

Time-dependent sources posed a particular threat on the calculators: a value like $10 e^{-t} \sin(2t + \pi/6)$, carried symbolically through every equation of a transient solve, could stall the CAS. The remedy, affectionately documented as the (*Impala*) mode [2]—named for the antelope that leaps over obstacles rather than running through them—exploits linearity. On entry, each complicated source value is replaced by a fresh placeholder symbol (“a nickname”); the system, now small and clean, is solved; and on the way out, the Laplace transform of the original value is substituted for the placeholder *in every answer*. Because the circuit is linear, every answer is linear in the source values, and the late substitution is exact. The mode was always automatic—“the (Impala) is always there...you have to do nothing at all”—waking

only when such a source was present.

Version 9 no longer needs the mechanism as an optimization: SymPy carries the expressions directly. But the idea—abstract the messy constants, solve, substitute back—remains a fine trick for any symbolic-computation setting, and its trace survives in how the modern TR pipeline still classifies each source value before deciding what, if anything, to transform (Algorithm 4).

3.5 Expert mode

Expert mode extends a circuit with user-supplied mathematics, and is where Symbulator turns from a calculator of answers into a design tool: instead of asking “what are the voltages, given every value?” one may fix an answer and solve for a value. Three extras are accepted, mirroring the prompts the calculator versions used:

- **Equations:** strings like `v_2 = 6` or `p_r1 = 0.025`, joined into Σ before solving;
- **Unknowns:** names to solve for—necessary when the symbol appears only in the *circuit* (a resistor of value `rx`), since the solver has no other way to know the symbol is to be varied rather than carried;
- **Conditions:** the calculator’s “|” (with) operator, in both of its meanings. An *equality* (`r_a = 1000`) is a substitution applied to the whole system at solve time—useful for trying numbers in an otherwise symbolic circuit. An *inequality* (`vs > 0`) is a restriction on which solutions may be returned, applied as a filter after the solve: exactly what `solve(...)|vs>0` did on the TI, and the natural way to select among the sign-symmetric roots a quadratic power constraint produces (see §5.2 for it in action). A restriction that excludes *every* solution is reported as such—the system solves, but the mathematics and the restriction disagree, which is an answer rather than a failure.

The original (*Expert*) mode was explicitly framed as making “a hole in that box”: it displayed the generated first-level equation set and unknowns list, and let the user read, modify, and keep them before the solve proceeded [2]. Its companion `solves()` tool—a general simultaneous-equation solver “based in the Symbulator core’s technology”—already had today’s shape: equations, unknowns in braces, and *conditional equations* (“when” values) applied as known substitutions, the direct ancestor of the `conditions` argument.

Algorithm 5 shows the folding, whose subtleties all stem from one theme: *an expert equation must constrain the circuit, never a lookalike variable*. Three cases require care.

First, an equation naming a known (`i_j1 = 5` on a 5 A source) would otherwise introduce a free variable that shares the name and satisfies itself; substituting $\mathcal{K}$ first turns it into a real constraint—or into a symbol-free residue, which is either a harmless tautology (dropped) or a flat contradiction (reported plainly).

Second, an equation naming a *third-degree* quantity (`r_e = 12000`: “the source must see 12 k Ω ”) is the mirror problem: those names are computed after the solve and are invisible to it. The remedy is to stamp the quantity’s defining equation into the system on demand—for the seen resistance, in the polynomial form $r_e \cdot (-i_e) = v_{n_1} - v_{n_2}$ so that a zero current cannot inject a division by zero—after which the constraint does real work. The definitions used here and the third-degree round of Algorithm 3 are the same formulas by construction, and the test suite holds them to that.

Third, symbols appearing only in expert equations are adopted as unknowns automatically (a convenience the calculator did not offer), so that defining a new quantity, `pout = v_2 * i_r2`, needs no separate declaration; an explicit unknowns list always wins over this inference.

Algorithm 5 EXPERT-AUGMENT: fold user mathematics into the system

Input: $\Sigma, \mathcal{U}, \mathcal{K}$; user equations Q , unknowns X , conditions C

```
1: for all name  $x \in X$  do
2:   add symbol  $x$  to  $\mathcal{U}$  if not present
3: end for
4: for all equation  $q \in Q$  (parsed; bare expressions read as  $q = 0$ ) do
5:   substitute  $\mathcal{K}$  into  $q$  ▷  $i_{j1} = 5$  must constrain, not alias
6:   if  $q$  has no symbols left then
7:     if  $q$  is false then error “contradicts the circuit” else drop  $q$  ▷ tautology
8:   else
9:     EMIT  $q$ 
10:  end if
11: end for
12: for all new symbol  $x$  appearing in  $Q$  but not in  $\mathcal{U}$  do
13:   if  $x$  names a third-degree quantity of some element then
14:     EMIT its defining equation; add  $x$  to  $\mathcal{U}$  ▷ see below
15:   else
16:     add  $x$  to  $\mathcal{U}$  ▷ a fresh derived quantity, e.g.  $p_{out}$ 
17:   end if
18: end for
19: for all condition  $x = v$  in  $C$ , in order do
20:    $\Sigma \leftarrow \Sigma[x := v]; \mathcal{K} \leftarrow \mathcal{K}[x := v];$  remove  $x$  from  $\mathcal{U}$ 
21: end for
```

3.6 Multiple solutions and their ranking

A circuit description alone yields a linear system with exactly one solution. Expert mode can change that: an added equation on a *power* is quadratic in the underlying unknowns, so a constraint like $p_{r_1} = 0.025$ on a circuit with a symbolic source may be met by $e = 5$ and $e = -5$ alike. Symbolator returns *all* solutions, ranked so the physically likely one leads. The ranking judges only the *design* unknowns—symbols that are not node voltages or branch currents, e.g. a component value being solved for—since voltages and currents are legitimately negative all the time. Solutions whose design unknowns are all real rank above any with complex values; among those, all-non-negative ranks above any negative; ties preserve the algebra system’s order. The first solution is presented as *the* answer, with the alternatives available to inspect.

Chapter 4

Tools on top of the template

The equivalence tools contain no new circuit physics. Each one *orchestrates* the solver of the previous chapters: it appends one or two *test* elements to a copy of the description, re-runs the ordinary analysis, and reads the desired figure off the answers. This design—tools as thin clients of one trusted engine—is worth imitating: every tool automatically works in every domain the engine supports, with symbolic values, dependent sources, op-amps, and expert extras, because it is the same code path throughout.

4.1 Thévenin and Norton equivalents

Algorithm 6 THEVENIN(D, n_1, n_2, d): equivalent seen from a port

```
1:  $\sigma_o \leftarrow$  solve  $D$  in domain  $d$  ▷ round 1: circuit as given (open port)
2:  $v_{th} \leftarrow \sigma_o[v_{n_1}] - \sigma_o[v_{n_2}]$ 
3: if  $v_{th} = 0$  then error “not active; use ER”
4: end if
5: try  $\sigma_s \leftarrow$  solve  $D + \mathbf{stest}, n_1, n_2$  in domain  $d$  ▷ round 2: port shorted
6:    $i_{no} \leftarrow \sigma_s[i_{stest}]$ 
7: on failure ▷ fallback: the short as a limit
8:   solve  $D + \mathbf{rtest}, n_1, n_2, x$ ;  $i_{no} \leftarrow \lim_{x \rightarrow 0^+} i_{rtest}$ 
9:   if the limit is unbounded then  $z \leftarrow 0$ ;  $p_{max} \leftarrow \infty$ ; return
10:  $z \leftarrow v_{th}/i_{no}$ 
11:  $p_{max} \leftarrow v_{th} i_{no}/4$  (dc)   or    $\frac{|v_{th}|^2}{8 \Re z}$  (ac, amplitude convention; 4 under RMS)
12: return ( $v_{th}, i_{no}, z, p_{max}$ )
```

Algorithm 6 runs two rounds: the circuit as given yields the open-circuit voltage v_{th} between the chosen terminals, and a copy with a test short across them yields the short-circuit (Norton) current i_{no} ; then $z = v_{th}/i_{no}$ and the matched-load maximum power follow. Two refinements matter in practice. If the shorted copy cannot be solved directly—which can happen when the short collapses part of the circuit degenerately—the tool falls back to physics: a short is the limit of a vanishing resistance, so it plants a resistor of symbolic value x and takes $x \rightarrow 0^+$. An unbounded limit is then a real answer, not a failure: the port holds its voltage at any current, the equivalent impedance is zero, and p_{max} is unbounded. Second, expert extras are passed to *both* rounds—correct for conditions and definitions, which mean the same thing in either round, though not for a measurement-style equation that holds only in the unshorted circuit, a caveat the tool documents.

4.2 Equivalent resistance of a passive network

For a source-free circuit, ER injects a 1 A test current source across the port and reports the resulting voltage—numerically equal to the equivalent resistance (DC) or impedance (AC):

$z_{\text{eq}} = (v_{n_1} - v_{n_2})|_{I_{\text{test}}=1}$. One round, no ratios.

4.3 Two-port parameter extraction

Algorithm 7 PORT($D, n_1, n_2, \text{family}$): two-port parameters in one solve

- 1: pick test sources by family, with *symbolic* values x_1, x_2 :
 - z, a, b : current sources at both ports ▷ open-circuit family
 - y : voltage sources at both ports ▷ short-circuit family
 - h/g : current at one port, voltage at the other
 - 2: $\sigma \leftarrow$ solve $D +$ tests in domain d ▷ one round
 - 3: read V_1, V_2, I_1, I_2 at the ports from σ
 - 4: **for all** parameter $p_{k\ell}$ of the family **do**
 - response at port $k|_{x_{\bar{\ell}}=0}$
 - 5: $p_{k\ell} \leftarrow \frac{\text{response at port } k|_{x_{\bar{\ell}}=0}}{x_{\ell}}$ ▷ zeroing $x_{\bar{\ell}}$ opens/shorts that port
 - 6: **end for**
 - 7: for a/b : extract z -parameters as above, then convert by the closed-form $z \rightarrow a$ ($z \rightarrow b$) matrix identities
 - 8: **return** $\{p_{11}, p_{12}, p_{21}, p_{22}\}$
-

PORT (Algorithm 7) characterizes a circuit as a two-port of any of the six families. Its trick is to make the test sources' values *symbolic*: with excitations x_1, x_2 left as free symbols, a single solve captures the port responses as linear functions of both, and each parameter is read off by substituting the *other* port's excitation to zero and dividing by this port's own—algebraically identical to the textbook's “open (or short) the other port and measure the ratio,” but with no second solve. The choice of source type per family mirrors the definitions: zeroing a test *current* opens a port, zeroing a test *voltage* shorts it, which is exactly the side condition each parameter family's definition demands. The chain families a and b are obtained from an intermediate z extraction through their standard conversion identities rather than a bespoke excitation.

4.4 Small tools

Three helpers round out the toolkit. **pr** is the parallel combination $(\sum_k 1/z_k)^{-1}$ with a dominant-zero rule, available both as a function and as the **[a,b]** value shorthand. **pf** reports the power factor $|\cos(\angle V - \angle I)|$ of a voltage–current phasor pair, labelled *leading*, *lagging*, or *in phase* by the sign of the angle difference. **gain** computes, from input and output voltage–current pairs, the voltage gain $A_v = v_2/v_1$, current gain $A_i = i_2/i_1$, power gain $A_p = \Re(-A_v \overline{A_i})$, and input impedance $Z_i = v_1/i_1$.

4.5 What the port to version 9 changed

Version 9 is a faithful port: its stamping rules, naming conventions, degree architecture, and tool behaviours were transcribed from the TI-Basic sources and validated against the printed answers of the version 7/8 documentation—322 worked examples, each run and compared. The genuine differences are few enough to enumerate, and each is instructive.

1. **Staging is now conceptual.** The level split was, on the calculator, an optimization that kept the simultaneous system within the hardware's reach. SymPy removes the

constraint, and the port solves the joined system at once, keeping direct substitution only where it is always locally computable (current-source and capacitor currents). Same physics, same answers, less machinery.

2. **Transforms come from the algebra system.** Versions 7/8 called Frederiksen's Laplace libraries; version 9 uses SymPy's `laplace_transform` and its inverse, with guards on both ends: a transform that comes back unevaluated is refused rather than propagated, and answers that fail to invert are omitted individually.
3. **One bug was fixed rather than preserved.** The original routed a 0 F capacitor through the zero-value short-circuit rule; a 0 F capacitor is an open circuit, and the port's admittance-form stamping gets this right with no special case.
4. **Op-amps are stamped, not limited.** The earliest versions solved op-amp circuits with a finite model and applied the CAS `limit()` command to every stored answer [1]—idealization as a literal limiting process. The port stamps the ideal (nullor) equations directly.
5. **Phasors become rectangular numbers on entry.** The calculators' CAS handled polar forms natively; SymPy carries $e^{j\theta}$ unevaluated in ways that can stall a mesh system indefinitely, so angles are converted at the gate—trading a vanishing amount of exactness for solves that reliably terminate.
6. **Ambiguity is surfaced, not guessed.** A bare suffix (1k) meant SI on the calculator; in a library consumed by applications, it is refused by default with a structured error listing every ambiguous token, and the caller chooses a policy.
7. **Expert mode reaches further.** The original barred expert extras from the equivalence tools and required every new unknown to be declared; the port passes extras through the tools' rounds and adopts new symbols automatically, with the explicit list still winning.

Chapter 5

Exemplars

Symbulator’s value shows best on circuits that are painful by hand and effortless once described. This chapter collects a few such exemplars, each solved by a single call.

5.1 The two-stage amplifier of 1999

The circuit that closed the original 1999 paper [1] was the author’s own final project for his Electronics II course: a two-stage small-signal amplifier model—source resistance, two RC loading networks, two voltage-controlled current sources ($0.05 v_1$ and $0.05 v_3$), coupling capacitors of picofarads against load resistors of kilohms—whose gain Bode diagram was requested over 10^4 to 10^{13} rad/s. In Symbulator notation:

```
e1,5,0,vg
r1,5,1,150  r2,1,0,1'k  cc1,1,0,100'p  cc2,1,2,3'p
jd1,2,0,0.05*v_1  r3,2,0,2'k
r4,2,3,100  r5,3,0,1'k  cc3,3,0,100'p  cc4,3,4,3'p
jd2,4,0,0.05*v_3  r6,4,0,2'k
```

An FD analysis returns every node voltage as a rational function of s ; the transfer function is one division, v_4/v_5 , and the Bode plot follows. On the 1999 TI-89 this took three minutes to solve and another 52 seconds to plot—against “around half an hour, or more” by hand, as any electrical engineering student of the era would attest [1]. On version 9 the same description solves in under a second. The circuit makes a fine exemplar because it exercises so much at once: dependent sources written exactly as the textbook writes them, mixed magnitudes across nine orders, and a symbolic source riding through the whole analysis untouched.

5.2 One of each: the 2013 showcase

The author composed this problem in 2013 with the express purpose of showcasing the powerful simplicity of Symbulator. The circuit contains one dependent source of each of the four species—a VCCS ($0.2 v_{R7}$), a CCVS ($0.1 i_{R5}$), a CCCS ($2 i_{R1}$), and a VCVS ($0.7 v_{R6}$)—woven through seven resistors, with an independent voltage source V_s and an independent current source I_s left symbolic. The question inverts the usual direction of analysis:

Find positive values of V_s and I_s such that the VCCS delivers 80 W and the CCVS dissipates 0 W.

By hand this is a genuinely painful problem: two simultaneous *power* constraints—each quadratic in the underlying unknowns—on a network whose every mesh is entangled with a controlled source. In Symbulator it is a description and one expert call. Naming the nodes **a–n** and the elements after their labels:

```

es,e,0,Vs   js,0,d,Is
r1,e,m,10   r2,a,e,20   r3,m,0,30   r4,b,m,40
r5,n,m,50   r6,c,d,60   r7,n,d,70
jd1,a,b,0.2*v_r7   ed2,c,b,0.1*i_r5
jd3,n,c,2*i_r1   ed4,0,n,0.7*v_r6

```

with expert equations $p_{jd1} = -80$ (the solver’s p is power *consumed*, so delivering 80 W is consuming -80), unknowns vs, is ,¹ and the positivity written directly as *conditions*: $is > 0, vs > 0$.

Every piece of Chapter 2 participates. The dependent values name answers of all three degrees (i_{R1}, i_{R5} are second degree; v_{R6}, v_{R7} third), so the known-reference closure (§2.4.2) rewrites them into node-voltage constraints. The expert equations name third-degree powers, so their defining equations are stamped in on demand (§3.5)—and being powers, they are quadratic, so the system has *four* solutions: two sign-symmetric pairs. The inequality conditions act exactly as the calculator’s $|$ operator did (§3.5): they filter the solve’s results, and the single call returns the one solution that survives:

$$V_s = \frac{3490}{\sqrt{39259}} \approx 17.614 \text{ V}, \quad I_s = \frac{551}{7\sqrt{39259}} \approx 0.39727 \text{ A}.$$

(Without the conditions, all four solutions come back, ranked with the all-positive pair first, per §3.6—either road reaches the answer; the conditions say what was meant.)

The solved circuit also explains *how* the zero-dissipation constraint is met, and the mechanism is elegant: in the winning solution $i_{R5} = 0$ exactly. The C CVS’s voltage is $0.1 i_{R5}$, so the constraint drove its own controlling current to zero—the source dissipates nothing because the circuit has arranged for it to be nothing, a fact one reads directly off the answer sheet ($i_{R5} = 0$, hence $v_{ed2} = 0$, hence $p_{ed2} = 0$) rather than divining it from the topology. This is the symbolic simulator’s characteristic gift: not just the numbers asked for, but the structure that produced them.

5.3 Prof. Eliane Boulet’s switching transient

This transient problem, from Prof. Eliane Boulet, is a classic of the two-switch genre. For $t < 0$, an 18 V source behind 8Ω feeds a resistive ladder ($12 \Omega, 18 \Omega, 6 \Omega$) holding two capacitors ($\frac{1}{6}$ F at node a , voltage v_1 ; $\frac{1}{3}$ F at node b , voltage v); a waveform source $10 e^{-t} \sin(2t + 30^\circ)$ A waits behind an open switch. At $t = 0$ both switches act: the DC source disconnects, the waveform source connects. *Find v_1 for $t > 0$.*

The problem is an exemplar because Symbolator solves *both* halves of it, and the halves chain. The $t < 0$ circuit is a DC analysis—capacitors open automatically at steady state (§2.4)—whose node voltages *are* the initial conditions:

```

e,s,0,18   r8,s,a,8   c1,a,0,1/6   r12,a,0,12
r18,a,b,18   r6,b,0,6   c2,b,0,1/3

```

returns $v_a = 9 \text{ V}$ and $v_b = \frac{9}{4} \text{ V}$. The $t > 0$ circuit drops the source branch, connects the waveform source, and carries those two numbers as the capacitors’ initial-condition fields—the fifth field that has existed since the five-column matrices of 1999:

¹The name `is` is worth a footnote of its own. When this chapter was first drafted, version 9 refused it: `is` is a reserved word of Python, and the port’s expression reader—which parses values through Python’s own grammar—let that refusal leak through to the circuits user, for whom `is` is the most natural name a source current could have. The drafting of this monograph exposed the wrinkle, and solver 0.5.19 removed it: Python keywords used as bare names are now shielded from the language’s parser and become ordinary symbols, so `is` works everywhere a name can appear.

```

c1,a,0,1/6,9  r12,a,0,12  r18,a,b,18
r6,b,0,6  c2,b,0,1/3,9/4
js,0,b,10*e^(-t)*sin(2*t+30*pi/180)

```

A TR analysis (§3.3) recognizes the source value as a waveform, transforms it in, solves in s with the initial conditions inside the capacitor stamps, and inverts every answer. The result, for $t \geq 0$:

$$v_1(t) = \frac{193 + 160\sqrt{3}}{34} e^{-t/2} + \frac{153 - 170\sqrt{3} + 20 \cos(2t + \frac{\pi}{6}) - 80 \sin(2t + \frac{\pi}{6})}{34} e^{-t}$$

$$\approx 13.83 e^{-t/2} - 4.16 e^{-t} + e^{-t} [0.588 \cos(2t + 30^\circ) - 2.353 \sin(2t + 30^\circ)] \text{ V.}$$

The structure is legible at a glance: the circuit's two natural modes ($e^{-t/2}$ and e^{-t}) and the forced response riding on the source's own e^{-t} envelope. And the answer carries its own check: evaluating at $t = 0$ gives exactly 9 V—the capacitor voltage is continuous across the switching instant, as physics demands. Done by hand, this problem is an afternoon of Laplace algebra with two initial conditions threaded through every step; with Symulator it is two descriptions and two calls, and the student's attention stays where Prof. Boulet presumably wanted it—on the switching, the steady states, and the modes.

5.4 Three-phase: wye–delta with line impedances

Three-phase problems are the traditional graveyard of sign errors: three sources at 120° intervals, line impedances, a delta load, and a page of phasor mesh algebra between the question and the answer. This one is Alexander & Sadiku's Example 12.11 [8], a balanced wye–delta system *with* line impedances—the variant textbooks save for last. In Symulator the sources are typed the way the book prints them, angle notation and all:

```

ea1,na1,0,100  eb1,nb1,0,(100∠−120°)  ec1,nc1,0,(100∠120°)
raa,na1,na2,1  rbb,nb1,nb2,1  rcc,nc1,nc2,1
rac,na2,nc2,100+24*pi*j  rcb,nc2,nb2,100+24*pi*j  rba,nb2,na2,100+24*pi*j

```

One AC call returns every line current and node phasor; in polar display, the a -line current is $i_{raa} = 2.350 \angle -36.20^\circ$ A and the load's line voltage $v_{na2} - v_{nb2} = 169.94 \angle 30.81^\circ$ V. The exemplary part is what is *absent*: no wye–delta transformation, no single-phase equivalent, no bookkeeping of which angle belongs to which mesh—the standard manoeuvres exist to make hand solution tractable, and a solver that simply writes all the equations does not need them. The student can instead spend the attention where the three-phase insight lives: on the $\sqrt{3}$ and 30° relationships visible between the answers.

5.5 The ideal transformer, symbolically

From the version 9 teaching corpus (after Alexander & Sadiku's Fig. 13.33): a source v_{s2} behind an impedance z_2 on the secondary of an ideal $1:n$ transformer, viewed from the primary terminals. Everything in the description is a name:

```

t,2,3,1,n  r2,3,4,z2  e2,4,0,vs2

```

Asking for the Thévenin equivalent seen at the primary port—TH at (2,0), Chapter 4—returns

$$v_{th} = \frac{v_{s2}}{n}, \quad z_{eq} = \frac{z_2}{n^2}.$$

That is, the tool *derives* the referred-source and referred-impedance identities every machines course teaches: a voltage refers by the turns ratio, an impedance by its square. When the answer to a symbolic question is the textbook’s own theorem, the simulator has done something a numeric tool cannot: it has produced knowledge in the same form the student is supposed to carry away.

5.6 Coupled coils with initial conditions

Composed for this monograph: two identical coils, $L_1 = L_2 = 2$ H, coupled with $M = 1$ H, each closed on its own $3\ \Omega$ resistor; no sources anywhere. The primary carries 1 A at $t = 0$; the secondary is at rest. What happens?

```
l1,1,0,2,1  r1,1,0,3
l2,2,0,2  r2,2,0,3  m,11,12,1
```

The initial condition rides in the inductor’s optional field, exactly where it has lived since the five-column matrices of 1999, and the FD stamps carry it as the $i(0)/s$ terms of §3.2—including inside the *mutual* terms, which is the part that is miserable to do by hand. One TR call:

$$i_{L1}(t) = \frac{e^{-t} + e^{-3t}}{2}, \quad i_{L2}(t) = \frac{e^{-t} - e^{-3t}}{2}, \quad v_2(t) = \frac{3(e^{-3t} - e^{-t})}{2}.$$

The physics is legible in the algebra: the circuit’s two modes are the *common* mode (currents aiding, effective inductance $L + M = 3$ H, $\tau = 1$ s) and the *differential* mode ($L - M = 1$ H, $\tau = \frac{1}{3}$ s), and the answers are exactly their sum and difference. The secondary current starts at zero, rises as the fast mode dies, peaks, and decays with the slow mode—energy handed across the coupling and returned to heat, the whole story in two exponentials.

5.7 Two op-amps, all symbols

From the teaching corpus (after Bobrow’s Example 3.3 [9]): a two-op-amp summing cascade in which every conductance is a symbol, g_1, g_2, g_3, g_4, g , and the source is v_s :

```
e,1,0,vs  r12,1,2,1/g1  r14,1,4,1/g2
r23,2,3,1/g  r34,3,4,1/g  r2o,2,o,1/g4  r4o,4,o,1/g3
o1,0,2,3  o2,0,4,o
```

Two nullors, seven symbolic elements, one DC call:

$$v_o = v_s \frac{g_1 - g_2}{g_3 - g_4}.$$

This is the answer a designer wants, in the form a designer wants it: the output is a *difference of conductances over a difference of conductances*, so the structure of the circuit is read directly off the expression—set $g_1 = g_2$ and the output nulls; let $g_3 \rightarrow g_4$ and the gain diverges; trim any one conductance and the sensitivity is a one-line partial derivative. A numeric simulation of the same circuit would have produced one number per run and none of this shape. Symbolic answers on op-amp networks are, in the authors’ experience, the single most persuasive demonstration of what Symbolator is *for*.

Chapter 6

The users, over the decades

Software for students lives or dies by whether students reach for it, and Symbulator’s story is best told in its users’ own words, collected over more than twenty-five years [1, 2, 6].

6.1 The early years

From the first year, the reports had a pattern: relief at the mathematics lifted away, and surprise that a calculator could do it. Chris Riegel, then president of the IEEE student chapter at the University of Kentucky, wrote: “For EE211 and EE221, Symbulator actually works better than SPICE because you get the results symbolically. Once you use Symbulator you’ll see what everybody has been talking about. If you’re not using Symbulator, you’re working too hard.” Jay Myers, of the University of Texas at Arlington, reviewed it at length: “Once a circuit is solved, it has all the information the user would want: node voltages, element currents, element power usage, voltage drops... It solves symbolically, unlike almost any other circuit simulator (even PC based). This program is, hands down, the best program for circuit analysis that I have seen.” [1]

The Spanish-language site gathered the same story from Latin America: thirteen users in one group at the University of Central Florida (“those who use all three programs always say yours is the easiest and fastest”); thanks from Chile “for making life easier for all of us who study Electrical Engineering”; congratulations from Mexico on a manual “perfectly well explained” [2]. Users reported from over twenty countries on five continents, and the program spread the way student tools spread: one classmate at a time.

6.2 The long run

A quarter century later, the collected praise reads like a single voice across generations [6]: “the masterpiece of TI-89 programming” (Alex Astashyn, New York); “Symbulator is and always will be the best program for TI-89/92/V200” (Charles Ware, Ohio); “it gives you everything you need... DC, AC, transient analysis, Thévenin, Bode plots... everything!” (Nikolaos Trichakis, Thessaloniki); “I wish I had it while going through school” (Joe Riel—the same Joe Riel whose Syrup had pointed the way); “Symbulator is worth the cost of a TI-89” (Pier Giorgio Raponi, Pisa); “it certainly is helping me to focus on my studying of EE instead of trying to remember all the math” (Jim Wachtel, Missouri). Several of these correspondents—Frederiksen, Hutcheson, Burkett, Riel among them—became collaborators, translators, and friends; the full roll is kept in the Symbulator documentation’s credits [6].

6.3 Closing reflections

The users’ words converge on the claim this monograph opened with, and the internal logic presented here explains *why* the tool earns them.

Symbols first. Because the engine is symbolic end to end, a student can solve the circuit *as drawn in the textbook*—with R_1 , L , and v_S as letters—and watch the structure of the answer:

which elements appear, which cancel, how a pole arises. A numeric simulator answers “what”; a symbolic one also answers “why,” and the difference is precisely the part a course wants students to internalize.

The input is the concept. The description language is a one-to-one transcription of a schematic: an element, its terminals, its value. Dependent sources are ordinary values that name an answer; degrees mirror how a hand solution actually proceeds (voltages, then currents, then powers). The equations generated are the ones a student would write. Nothing in the interface is an artifact to learn and later unlearn.

Failure is didactic. The solver’s refusals teach: it names the loop of sources that contradicts itself, the node whose current sources cannot balance, the floating sub-circuit with no reference. The same care appears at the input (an ambiguous `1k`, quoted back exactly as typed) and in the tools (an unbounded short-circuit current reported as the real answer that it is). Error paths designed this way convert the classic frustrations of simulation into circuit-theory lessons.

Verification at scale. Because answers are exact, the whole of Symbulator’s teaching corpus is machine-checkable against the documentation’s printed answers—and is checked, example by example. A student can trust that the tool and the book agree, and an instructor can extend the corpus knowing that regressions announce themselves.

There is also a lesson in the project’s longevity. The same internal logic has now run on 1990s calculator hardware and on a modern computer algebra system, with the staged degree architecture surviving as a *conceptual* account even where it is no longer an operational necessity. Designs that mirror the structure of the subject matter—here, of circuit analysis as actually taught—age well; the hardware around them is what changes. And the most recent chapter, a two-decade-old port completed in weeks by its author working with an AI assistant, suggests that long-lived educational software can be carried to each new platform at a fraction of its original cost, so long as its logic was worth preserving in the first place. *The purpose of circuit analysis is insight, not mathematics*—and the purpose of Symbulator, across every platform it has lived on, has been to keep the two in their right order.

Acknowledgments

Symbulator 9 is built on Python and SymPy, and the authors thank their creators and maintainers. Versions 7 and 8’s Laplace transforms are due to Lars Frederiksen, adapted for the TI-Nspire by Philippe Fortin. Over the decades, many users improved Symbulator through suggestions and corrections; they are acknowledged individually in the Symbulator documentation. The teaching corpus draws its problems from widely used circuit analysis textbooks, credited in full in the documentation.

Bibliography

- [1] R. Perez-Franco (under the pseudonym *Thévenin*), “Symbulator: simulador simbólico de circuitos lineales para calculadoras,” competition paper, IEEE Student Paper Contest for Latin America, Nov. 1999 (first place, 2000). English translation: “Symbulator: the calculator-based symbolic circuit simulator.” Both preserved in this monograph’s **references/** archive.
- [2] R. Perez-Franco, *Symbulator: Symbolic Circuit Simulator — on-line documentation*, 2001 website (introduction, instructions, expert-mode, Impala-mode, tools, and examples pages), documenting Symbulator Q (version 5) and *The Symbulator Book* (12 chapters, in Spanish; chs. 1–4 translated by T. Hutcheson and D. Burkett). Preserved in this monograph’s **references/** archive.
- [3] R. Perez-Franco, *The Symbulator Book*, unfinished English edition, ca. 2014, documenting Symbulator 6. Preserved in this monograph’s **references/** archive.
- [4] R. Perez-Franco, “Symbolic simulation of linear circuits with Symbulator,” unpublished fragment, ca. 2014. Preserved in this monograph’s **references/** archive.
- [5] R. Perez-Franco, *Symbulator: symbolic simulation of linear circuits*. [Online]. Available: <https://symbulator.com>
- [6] R. Perez-Franco, *The Symbulator 9 Documentation*. [Online]. Available: <https://learn.symbulator.com>
- [7] A. Meurer *et al.*, “SymPy: symbolic computing in Python,” *PeerJ Computer Science*, vol. 3, p. e103, 2017.
- [8] C. K. Alexander and M. N. O. Sadiku, *Fundamentals of Electric Circuits*, 7th ed. New York: McGraw-Hill, 2020.
- [9] L. S. Bobrow, *Elementary Linear Circuit Analysis*, 2nd ed. Oxford: Oxford Univ. Press, 1987.
- [10] R. E. Thomas, A. J. Rosa, and G. J. Toussaint, *The Analysis and Design of Linear Circuits*, 5th ed. Hoboken: Wiley, 2006.